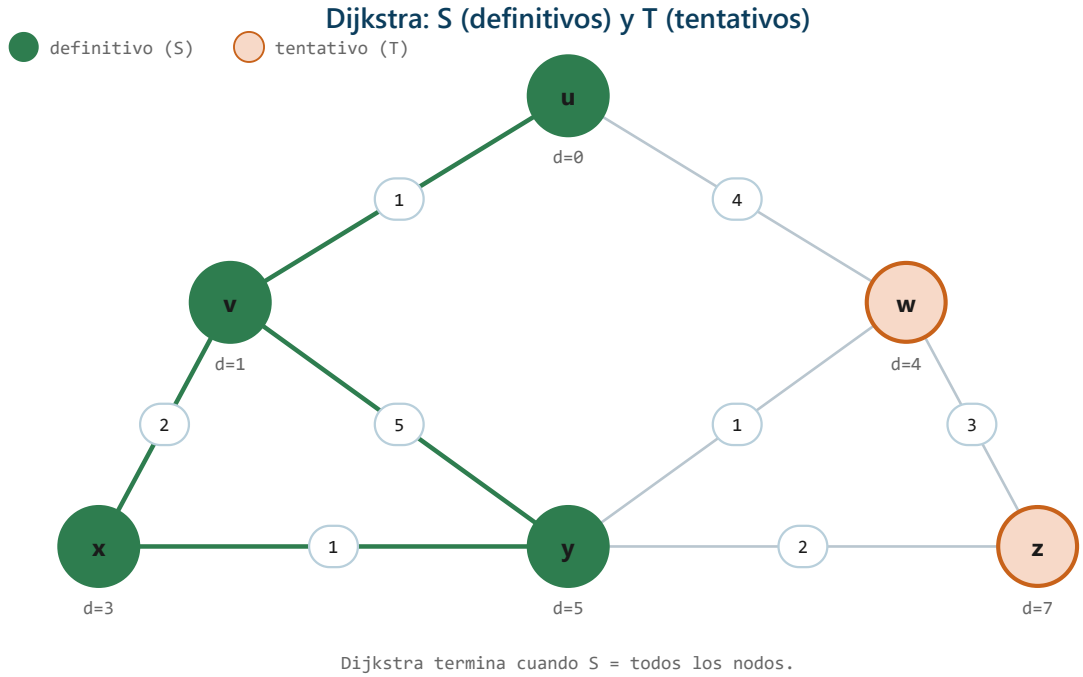
Clase 33 — Enrutamiento link-state, Dijkstra y OSPF



Bloque: D (Capa de red) · **Semana:** 11 · **Duración estimada:** 120 min **Correlativa:** Clase 32 (distance-vector: Bellman-Ford, conteo a infinito, RIP) · **Sigue con:** Clase 34 (BGP y el enrutamiento entre operadores) **Material de referencia:** Tanenbaum cap. 5 §5.6 · RFC 2328, RFC 5340, RFC 5308

🎯 Al terminar esta clase vas a poder

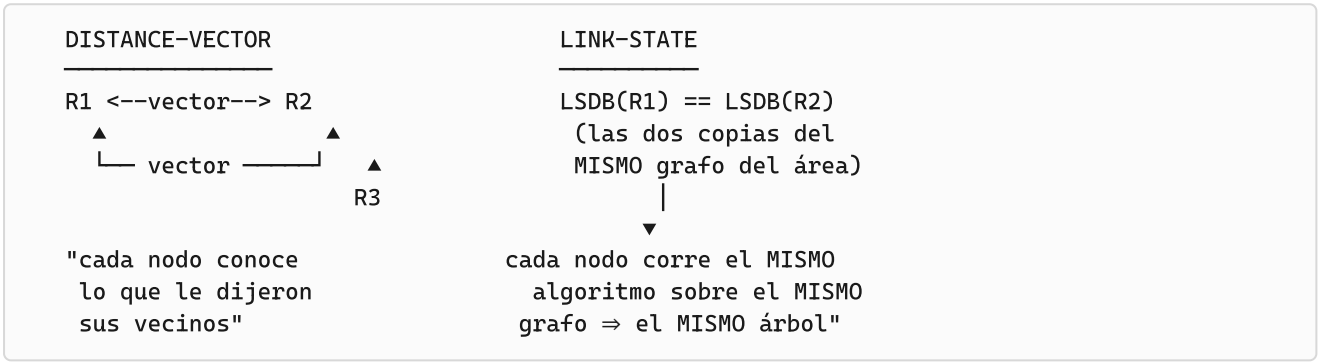
1. Explicar en qué se diferencia **link-state** de **distance-vector** con una tabla comparativa de doce filas.
2. Describir un **LSA**, hacer el **flooding** de uno paso a paso y justificar cómo la **edad** y el **número de secuencia** evitan el bucle.
3. Escribir el **algoritmo de Dijkstra** y explicar **por qué funciona** (principio de optimalidad).
4. **Ejecutar Dijkstra a mano con la tabla de iteraciones**, con los nodos definitivos, los tentativos y el vector de distancia.
5. Nombrar y explicar los **cinco mensajes de OSPF**, con sus tiempos, su TTL y sus direcciones de destino.
6. Justificar por qué existen las **áreas**, qué son los **LSA de tipo 3 y 4**, y **por qué se elige un DR/BDR** en una red multiacceso.
7. **Demostrar con números** que la métrica de hops puede elegir el camino pésimo, y resolver el `2009-12 P3`.
8. Analizar la **oscilación de métrica** entre enlaces paralelos del `2007-03 Problema 2` y proponer la medición de ancho de banda que la evita.

> **Prerequisito del examen:** `2009-02 Problema 2` (30 pts), `2007-03 Problema 2` (30 pts), `2009-12 Pregunta 3` (8 pts), `2007-12 Pregunta 1` (5 pts).
>
> ⚠️ **Nota sobre el código de un examen.** El enunciado "Explique resumidamente los 5 mensajes que se implementa en OSPF" (5 puntos) **no es** `2007-03 P1`: en `eirc0703` / `sirc0703` la Pregunta 1 es "*Describa el modelo de capas TCP/IP*". El examen que pregunta exactamente eso es **`eirc0712` / `sirc0712`, Pregunta 1** (27 de diciembre de 2007). Lo resolvemos en §3.4 con el código correcto. Además, el PDF de `sirc0712` **no publica la solución de las preguntas teóricas**, así que §3.4 va marcada como *(solución propia, contrastada con RFC 2328)*.

PARTE 1 — TEORÍA

1.1 La idea en una frase

> **En distance-vector cada router cree que el mundo es como le dijeron sus vecinos. En link-state cada router tiene un mapa del mundo y corre el mismo cálculo que sus vecinos.**



1.2 La tabla comparativa de doce filas

#	Aspecto	**Distance-vector**	**Link-state**
1	Qué conoce cada router	Solo sus vecinos directos	**Toda la topología del área**
2	Qué mensaje intercambia	Un **vector de distancias** (su tabla entera)	Un **LSA** (el estado de **un** enlace)
3	Tamaño del mensaje	Crece con el número de destinos	**Constante**: el tamaño de un enlace
4	Conocimiento topológico	**No existe** en el sistema	Existe y es **idéntico** en todos los routers del área
5	Algoritmo	**Bellman-Ford**	**Dijkstra** (SPF)
6	Periodicidad	Temporizador (en RIP 30 s)	**Evento**: el flooding es inmediato al cambiar
7	Problema clásico	**Conteo a infinito** (clase 32)	**Ninguno**: los caminos convergen al árbol más corto
8	Convergencia	**Lenta**: proporcional al número de saltos	**Rápida**: proporcional al **log** del número de routers
9	Ciclos / bucles	Necesita **split horizon**, *poison reverse*, *hold-down*	**No los necesita**: el árbol no tiene ciclos por construcción
10	Reacción ante un enlace caído	La noticia tarda hasta el temporizador y los routers se realimentan	El LSA del enlace se inunda **ya** y todos recalculan
11	Memoria y CPU	**Bajo**: tabla chica, cálculo simple	**Alto**: LSDB completa y SPF costoso, de ahí las **áreas**
12	Escala	LAN o campus chico (RIP: 15 saltos)	**Red corporativa, backbone de operador, datacenter** (OSPF, IS-IS)

> **Las filas 9 y 12 van juntas:** link-state no necesita split horizon *precisamente porque* paga el precio de la fila 11. Un mecanismo no se borra gratis: se cambia por otro.

1.3 El LSA: qué es y qué lleva adentro

Un **LSA (Link State Advertisement)** describe **el estado de los enlaces de un router**, no sus rutas.

```
┌──────────────────────────────────────────────────────┐
│  LSA  (cabecera común a todos los tipos)             │
│  ├─ LS Age (edad)            ← cuánto vive este LSA  │
│  ├─ Options                  ← capacidades del router│
│  ├─ LS Type                  ← 1..5                  │
│  ├─ Link State ID            ← quién lo origina      │
│  ├─ Advertising Router       ← el Router ID          │
│  ├─ LS Sequence Number       ← control de frescura   │
│  ├─ LS Checksum              ← detecta corrupción    │
│  └─ Length                                           │
│  Cuerpo. Para el TIPO 1 (Router-LSA):                │
│  ├─ Link ID 1  ─┐                                    │
│  ├─ Link Data 1  ├─ (interfaz, tipo de red, MASK,    │
│  ├─ Link Type 1 ─┘  costo hacia el vecino)           │
│  ├─ Link ID 2 ...                                    │
│  └─ Link ID n ...   ← un par POR CADA ENLACE         │
└──────────────────────────────────────────────────────┘
```

Lo clave: **el LSA no dice "a `x` se llega a 3 hops"**. Dice **"del router `u` al router `v` hay un enlace y mi costo es 2"**. Es un hecho local, verificable, y por eso no puede ser un rumor: o el enlace está o no está.

El LSDB (Link State Database) es el conjunto de todos los LSA del área que tiene un router. **La condición de sincronización es simple y es toda la garantía del protocolo:**

```
LSDB(R1)  ==  LSDB(R2)  ==  LSDB(R3)  ==  ...  ==  LSDB(Rn)
```

Si dos routers del área tienen **el mismo grafo**, entonces **calculan el mismo árbol**, aunque lo calculen en instantes distintos. Distance-vector no puede decir esto: sus entradas son distintas porque las estimaciones nacen de rumores distintos.

Tipo	Nombre	Qué describe	Quién lo genera
1	Router-LSA	Los **enlaces del router** (vecinos, costos, máscaras)	Todo router del área
2	Network-LSA	Los routers **conectados a una red multiacceso**	El **DR/BDR** (§1.7)
3	Summary-LSA	Un **prefijo** y su costo desde el ABR hacia él	Un **ABR** (§1.7)
4	ASBR-Summary-LSA	Cómo llegar al **ASBR** que anuncia externos	Un **ABR**
5	External-LSA (ASE)	Una ruta **de otro sistema autónomo**, con métrica de tipo externo	Un **ASBR** (redistribuidor)

> El tipo 5 conecta esta clase con la 34: los LSA de tipo 5 son **rutas que vienen de BGP**. Por eso son de tipo externo y su costo no es comparable con el de los enlaces del área.

1.4 El flooding, paso a paso, y los tres controles

Flooding (*inundación*) es la propagación de un LSA a **todos** los routers del área. Ya lo vimos como mecanismo de reenvío en la Clase 24 §1.6; acá cambia **quién** lo hace y **con qué** lo controla.

```
Paso 0: R3 detecta que el enlace (R3,R5) costaba 7 y ahora cuesta 70.
        Genera un LSA nuevo:  (LS id = R3, seq = 0x2000000E, age = 0)
                                                          |
Paso 1: R3 lo manda por CADA interfaz, menos por la       ▼
        que lo recibió.
   ┌──────┬──────┬──────┬──────┐
   ▼      ▼      ▼      ▼      ▼
  R1     R2     R4     R5     R6        (y R3, que lo originó)

Paso 2: cada vecino que lo recibe hace TRES preguntas:
           ¿ya lo tengo?
                ├─ sí, misma instancia ──► DESCARTO. No reenvío. (fin)
                ├─ sí, pero más viejo  ──► lo actualizo y REENVÍO
                ├─ sí, pero más nuevo  ──► lo ignoro (era un atraso)
                └─ no                   ──► lo instalo y REENVÍO

Paso 3: R1, que ya lo tenía viejo, lo actualiza y lo reenvía por
        TODAS sus interfaces menos por donde lo recibió.
        R2 lo recibe de R1 y ya lo tiene → DESCARTA.

Resultado: en el grafo Steiner del flooding hay un árbol: O(N)
           reenvíos, no O(N²).
```

```
(1) NÚMERO DE SECUENCIA  (32 bits con signo, declarado)
    InicialSequenceNumber = 0x80000001  (el más negativo)
    MaxSequenceNumber     = 0x7FFFFFFF  (el más positivo)
    Rango negativo = "recién creado"; positivo = "estable".
    Cuando hay que actualizar y se está en 0x7FFFFFFF, se hace
    "flush" (se inunda el LSA viejo con MaxAge) y se arranca en
    0x80000001. Es el wrap de 32 bits con signo. ¿Para qué? Para
    que "nuevo" sea un orden TOTAL, no ambiguo.

(2) EDAD  (LS Age, en segundos; MaxAge = 3600 s)
    Todo LSA tiene un reloj que corre. A MaxAge se lo considera
    EXPIRADO y se tira de la base. Cuando el origen detecta que
    perdió la vista del LSA, lo inunda con Age = MaxAge: eso es
    el "flush", la forma limpia de "borrar" sin mensaje de borrado.
```

Y el tercer control, que es del flooding y no del LSA: un router **no reenvía por la interfaz por la que recibió** el LSA. Los tres juntos:

```
¿por qué no vuelve para atrás?    → NO se reenvía por la interfaz de
                                    entrada (enlace punto a punto, o
                                    tabla de vecinos en multiacceso)
¿por qué no vuelve por otro lado? → el que ya lo tiene NO lo reenvía
¿y si llega un LSA viejo?         → se ignora y no se reenvía
```

> **Dato para el examen:** los tres son necesarios. Sin el primero, dos routers de una línea punto a punto se pisan. Sin el segundo, el LSA da vueltas por un anillo. Describí los tres controles y el para qué de cada uno.

El flooding confiable: los tres mensajes que lo sostienen

El flooding puro es fiable pero **desperdicia ancho de banda** en redes grandes. OSPF lo hace **fiable con acuse de recibo**, y esa es la razón de existir de tres de los cinco mensajes:

```
Flujo normal:      LSU  ──────────────►  vecino
                   ◄──────────────  LSAck

Recuperación: el vecino NO acusa recibo (se perdió el LSU):
                   ◄─────  LSU, reenviado
                   ─────►  LSAck

"Me falta": el vecino tiene algo que yo no tengo:
                   ─────►  LSR  (pide ESA LSA específica)
                   ◄─────  LSU (con la LSA pedida)
                   ─────►  LSAck
```

Es una **ventana deslizante sobre el LSDB**: el origen manda hasta `N` LSAs pendientes de acuse; si no llegan en `RxmtInterval` (5 s), reenvía; si tras `N` intentos no consigue nada, **desarma la vecindad**.

1.5 Dijkstra: el algoritmo y por qué funciona

```
 1  Initialization:
 2      N' = {u}
 3      for all nodes v
 4          if v adjacent to u
 5              then D(v) = c(u,v)
 6          else D(v) = ∞
 7
 8  Loop
 9      find w not in N such that D(w) is a minimum
10      add w to N'
11      update D(v) for all v adjacent to w and not in N:
12          D(v) = min( D(v), D(w) + c(w,v) )
13          /* new cost to v is either old cost to v or known
14             shortest path cost to w plus cost from w to v */
15  until all nodes in N'
```

- **Líneas 3 a 6 — la inicialización.** `D(v)` es "mi **mejor estimación** del costo de llegar a `v`". Los vecinos directos se inicializan con el costo **real y directo** del enlace, que conozco sin preguntar a nadie. Todo lo demás arranca en `∞`.
- **Línea 9 — el mínimo sobre los que NO están en `N'`.** `N'` es el conjunto de nodos ya **definitivos**: su distancia ya no va a cambiar.
- **Línea 10 — se vuelve definitivo.** Y acá está **la clave de por qué funciona**: en cuanto `w` entra a `N'`, su `D(w)` **ya no puede mejorar**, y por lo tanto **nunca más vamos a necesitar recalcular usando `w` como punto de paso**. Por eso la línea 11 filtra "para todos los `v` adyacentes a `w` **y que no están en `N'`**": si `v` ya fuera definitivo, actualizarlo desde `w` no lo mejoraría. Los dos filtros juntos son la **invariante**: "`N'` contiene exactamente los nodos cuya distancia ya es óptima".
- **Línea 12 — la relajación.** `v` mejora si pasar por `w` sale más barato que llegar por lo que ya tuviera.
- **Línea 15 —** en un grafo conexo son `V − 1` iteraciones (más la inicialización).

Por qué funciona: el principio de optimalidad, demostrado

> **Principio de optimalidad (Bellman, 1957).** Si el camino más corto (con costos no negativos) de `u` a `v` pasa por `w`, entonces el tramo de `w` a `v` es él mismo un camino más corto desde `w`.

Demostración por contradicción, que es lo que hay que escribir en el examen:

Supongamos que el algoritmo elige `w` porque `D(w)` es el mínimo, y supongamos que existe un camino más barato a `w` que no pasa por `w` todavía en `N'`. Ese camino debe salir de `N'` por **algún** nodo `z` que ya es definitivo. Pero entonces ese camino cuesta

```
costo(u → … → z) + costo(z → … → w)
     ≥  D(z)  +  c(z,w)  ≥  D(z)  ≥  D(w)
```

porque `D(z)` es óptimo (invariante), porque `D(z)` era el mínimo cuando `z` entró a `N'`, y porque `c(z,w) ≥ 0`. O sea: **el camino más barato a `w` no puede ser más barato que `D(w)`**. `D(w)` es óptimo y `w` puede ser definitivo. ■

Y acá está la diferencia con Bellman-Ford: Dijkstra **exige `c(z,w) ≥ 0`**. Si un costo fuese negativo, el último paso de esa cadena (`+ c(z,w)`) podría **bajar** el valor y la desigualdad se rompe. Por eso **Dijkstra no funciona con costos negativos**: no es que Bellman-Ford sea "mejor", es que su condición es más débil.

```
Dijkstra      : costos ≥ 0  →  correcto y RÁPIDO (greedy, una pasada)
Bellman-Ford  : costos ≥ 0  →  correcto y LENTO (relaja V−1 veces)
Bellman-Ford  : costos < 0  →  correcto (es el único que puede)
Dijkstra      : costos < 0  →  INCORRECTO (greedy congela un nodo malo)
```

> **Para escribir en el examen:** no escribas "Dijkstra encuentra el camino más corto". Escribí "*Dijkstra mantiene la invariante de que el conjunto `N'` contiene exactamente los nodos cuya distancia mínima ya es conocida y que no va a cambiar, y la mantiene porque toda actualización ocurre sobre un nodo no definitivo; termina cuando `N' = V`, momento en el que todas las distancias son óptimas*". Eso es un 9-10. La frase de una línea es un 6.

La anatomía de la tabla de iteraciones — sin ella no hay puntos

```
┌──────┬──────────────┬──────────────────────────────────────────┐
│ Paso │ N' (definit.)│ D(v), D(w), D(x), D(y), D(z)             │
│      │              │ (con el predecesor entre paréntesis)     │
├──────┼──────────────┼──────────────────────────────────────────┤
│  0   │ u            │ 2,u   5,u   1,u    ∞     ∞             │
│  1   │ u x          │ 2,u   4,x   –     2,x    ∞             │
│  2   │ u x y        │ 2,u   3,y   –     –      4,y           │
│  3   │ u x y v      │ –     3,y   –     –      4,y           │
│  4   │ u x y v w    │ –     –     –     –      4,y           │
│  5   │ u x y v w z  │ (fin)                                    │
└──────┴──────────────┴──────────────────────────────────────────┘
       ▲
       └── el nodo que ENTRA en esta iteración
```

Tres convenciones que hay que declarar antes de empezar (escribilas, son puntos):

1. `N'` es el conjunto de nodos **definitivos**. Los nodos ya no se anotan en la columna de `D`, porque su valor **no cambia más**.
2. `D(n), p` = distancia mínima a `n` y **su predecesor**, que es lo que después arma el shortest-path tree y la tabla de forwarding.
3. Se **marca con una flecha** el nodo que entra en cada paso. Es la prueba de que no estás inventando la tabla.

1.6 Complejidad y crecimiento del estado — por qué existen las áreas

```
Bellman-Ford, versión matriz:      V rondas × V columnas    = O(V³)
Bellman-Ford, versión lista:       V rondas × E aristas     = O(V·E)
Dijkstra, matriz (min sobre V):    V pasos   × V búsqueda   = O(V²)
Dijkstra, cola de prioridad:       E relajaciones × log V   = O((V+E)·logV)
```

Red	Routers	Bellman-Ford ($O(V^3)$)	Dijkstra ($O(V^2)$)	Dijkstra con heap
Laboratorio	10	1.000	100	~30
Campus	100	1.000.000	10.000	~700
Operador grande	10.000	10^{12}	100.000.000	~1.400.000

Con `V = 10.000`, Bellman-Ford por matriz es **una billonada** de operaciones por cada cambio de enlace. Ni un procesador dedicado a routing aguanta eso.

```
Sin áreas:  todo router guarda una LSA por cada enlace de cada router
──────────────────────────────────────────────────────────────────
Si hay N routers y cada uno tiene, digamos, 4 enlaces:
    número de LSAs en el LSDB de CADA router  =  4·N
    tamaño del LSDB                           =  O(E)  =  O(N)

Con N = 20.000 routers y 4 enlaces cada uno:
    80.000 LSAs por router × ~100 bytes  =  ~8 MB por router
    ... y eso en los 20.000 routers que los tienen TODOS:
    ~160 GB de memoria solo en LSDB si nadie jerarquiza.

──────────────────────────────────────────────────────────────────
Y lo peor no es la memoria: es el TRÁFICO DE INUNDACIÓN.
Un cambio de enlace genera 1 LSA que viaja por todo el árbol.
Si ese cambio se reacomoda cada 30 segundos (una ruta que oscila)
× 80.000 routers × 100 bytes:  ≈ 2,3 Gbit/s de flooded por UN enlace.
```

El flooding es **O(N) por evento**; con muchos cambios simultáneos, **O(N × cambios)**. Esa multiplicación es lo que las áreas matan.

1.7 Los cinco mensajes de OSPF

Ésta es la parte que entra como pregunta de 5 puntos.

#	Mensaje	Código	Para qué	Cuándo	TTL	Destino
1	**Hello**	**1**	Descubrir y **mantener vivos** a los vecinos; negociar los intervalos; elegir el **DR/BDR**; descubrir el Router ID del vecino	**Cada 10 s** (p2p y broadcast); 30 s en NBMA/P2MP	**1**	`224.0.0.5` y `224.0.0.6`
2	**Database Description** (**DBD**)	**2**	En el arranque, **describir el contenido** del LSDB propio para que el vecino diga qué le falta	Arranque y refresco cada 30 min	**1**	unicast al vecino
3	**Link State Request** (**LSR**)	**3**	**Pedir explícitamente** una LSA que falta	Al completar la base, y en cualquier momento	**1**	unicast al vecino
4	**Link State Update** (**LSU**)	**4**	**Inundar** LSAs nuevas o actualizadas	Al flooding y al arranque	**1**	`224.0.0.5` o unicast
5	**Link State Acknowledgment** (**LSAck**)	**5**	**Confirmar** la recepción	Por cada LSU recibido	**1**	unicast al vecino

```
┌─ (a) TIMING ──────────────────────────────────────────────┐
│  HelloInterval = 10 s ; DeadInterval = 40 s (= 4 × Hello)  │
│  Por qué 40 y no 10: para withstand UNA pérdida de un Hello sin│
│  declarar muerto al vecino. Con 4× se toleran 3 pérdidas.  │
├─ (b) TTL = 1 ─────────────────────────────────────────────┤
│  OSPF corre directamente sobre IP (protocolo 89), NO sobre UDP │
│  ni sobre TCP: es un protocolo de capa de red, punto a punto.  │
│  TTL = 1: el paquete morre en el primer router. Un Hello es un │
│  MENSAJE DE VECINDAD LOCAL: su único destinatario válido es el │
│  router del otro lado del cable. Si alguien lo reenvía, hay un │
│  router mal configurado en el medio. El TTL=1 no es una       │
│  optimización: es una barra de seguridad.                     │
├─ (c) DIRECCIONES MULTICAST ──────────────────────────────┤
│  224.0.0.5 = AllSPFRouters → todos los routers OSPF           │
│  224.0.0.6 = AllDRouters   → solo el DR y el BDR              │
│  Los dos Hello van a 224.0.0.5 SIEMPRE; a 224.0.0.6 solo      │
│  cuando uno se postula a DR o BDR.                            │
│  224.0.0.0/24 está reservada para que un paquete no pueda     │
│  salir del segmento: el mismo truco que ff02::/16 en IPv6.    │
└──────────────────────────────────────────────────────────┘
```

> **Protocolo IP 89** (ospf = 89) es el número que va en el campo *Protocol* de la cabecera IP. Memorizalo junto con el **179** de BGP, de la clase siguiente.

El intercambio completo de los cinco

Ojo con la distinción clave: el Hello es periódico, los otros cuatro son del arranque.

```
R1 (Router ID 1.1.1.1)                          R2 (Router ID 2.2.2.2)

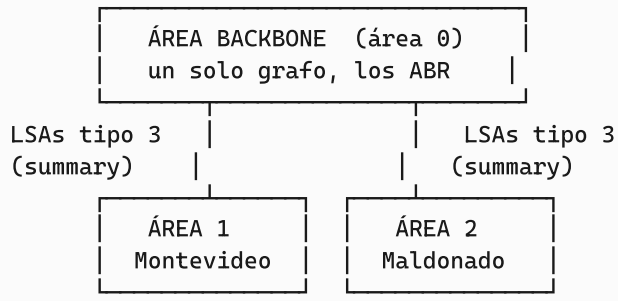
       │  ──── 1 Hello (cada 10 s, TTL 1, a 224.0.0.5) ────►│
       │  ◄──── 1 Hello ──────────────────────────────────────│
       │        R1 ve que su Router ID es el MAYOR → se postula DR
       │        R2 ve que el de R1 es mayor → se hace DROther
       │
       │  ──── 2 Database Description ──────────────────────►│
       │        "tengo 3 LSAs: uno mío, uno de la red, uno externo"
       │  ◄──── 2 Database Description ──────────────────────│
       │        "yo tengo 5: los 3 tuyos + 2 míos"
       │        R1 compara: le faltan los 2 LSAs de R2
       │
       │  ──── 3 Link State Request ──────────────────────►│
       │  ◄──── 3 Link State Request ──────────────────────│
       │  ◄──── 4 Link State Update ──────────────────────│
       │  ──── 4 Link State Update ──────────────────────►│
       │  ──── 5 LS Acknowledgment ──────────────────────►│
       │  ◄──── 5 LS Acknowledgment ──────────────────────│
       │
       │        AHORA: LSDB(R1) == LSDB(R2). Vecindad en estado FULL.
       ▼        El bucle 3-4-5 se repite por cada LSA que falte.   ▼
```

La máquina de estados de la vecindad (para dibujarla si te la piden):

```
Down —(Hello recibido)→ Init —(tu Router ID en su lista)→ 2-Way
                           ▼
                        ExStart —(elige DR/BDR)→ Exchange
                           ▼
                        Loading —(ya no falta nada)→ Full
En cualquier estado: si expira el DeadTimer → Down.
En 2-Way multiacceso: solo el DR pasa a Exchange; los DROther quedan.
```

Las áreas y los LSA de tipo 3 y 4

Si el LSDB completo de una red de 20.000 routers es insostenible, la solución es **no calcular el grafo entero**.




```
Cada área tiene SU PROPIO LSDB y corre SU PROPIO Dijkstra.
Los ABR pertenecen a dos áreas y se cruzan anunciando un LSA
de TIPO 3. El área 1 NUNCA ve un LSA del interior del área 2.
```

Por qué funciona, en dos frases: (1) **la exactitud se preserva dentro del área**: entre dos routers del área 0, Dijkstra sigue dando el camino más corto entre todos los del área 0, y los prefijos de las otras áreas se agregan después como un LSA de tipo 3; (2) **lo que se sacrifica es la optimalidad global**, y es un compromiso consciente: el camino entre el área 1 y el área 2 pasa obligatoriamente por el backbone.

	Tipo 3 — Summary-LSA	Tipo 4 — ASBR-Summary-LSA
Qué anuncia	Un **prefijo IP** y el costo desde el ABR que lo anuncia	Cómo llegar al **ASBR** (el que hace *redistribución*)
Quién lo genera	Un **ABR**	Un **ABR**
Para qué	Para que el área destino **no** necesite conocer el interior de la otra	Para poder **costear** el camino hacia una ruta externa que el área no contiene

```
tipo 3 dice:  "la red 192.0.2.0/24 está a costo 7 DESDE MÍ"
tipo 4 dice:  "el ASBR que anuncia las externas está a costo 4 DESDE MÍ"

Sin el tipo 4, un router del área 0 que recibe un tipo 5 no puede
saber qué costo poner al camino hacia ese ASBR, que no está en
su área. El tipo 4 es el puente que permite costear el prefijo
externo usando el grafo del backbone.
```

> **Concepto que vale puntos:** los LSA de tipo 3 son **la implementación del jerarquizado**. Pregunta clásica: "*¿qué pasaría si OSPF no tuviera áreas?*" → el LSDB y el SPF serían **O(tamaño de la red)** en vez de **O(tamaño del área)**, y el flooding de un cambio local impactaría a **toda** la red.

⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - `RFC 2328` (**OSPF**, 1998) es la especificación vigente de **OSPFv2**. - Los **cinco mensajes** con sus códigos 1 a 5, sus intervalos (10 s / 40 s), el **TTL = 1** y los **multicast 224.0.0.5 y 224.0.0.6**. - El algoritmo es **Dijkstra/SPF**, el LSDB se sincroniza, el flooding es fiable con LSR/LSU/LSAck. - **Áreas**, **backbone (área 0)**, **ABR**, **ASBR**, LSA de **tipo 1 a 5**, elección de **DR/BDR** en redes multiacceso. - El **TLV de costo de interfaz** y el concepto de *reference bandwidth*. - **IS-IS** (ISO 10589) es el otro IGP link-state que se despliega masivamente, sobre **CLNS**, directamente en capa 3 sin IP.

Cambió, o hay que matizar:

- **OSPFv3 (`RFC 5340`, 2009)** es el equivalente para **IPv6**: mismos mecanismos, mismos cinco mensajes, mismas áreas, mismos DR/BDR, pero independiente de IPv4. **No es una reescritura**: es un protocolo paralelo. Lo único que cambia de verdad es el direccionamiento y que **cada interfaz necesita su propio Router ID** (ya no se puede tomar "la IP más alta de la interfaz", porque en IPv6 las direcciones son efímeras).
- **IS-IS para IPv6**: `RFC 5308` define las extensiones de IS-IS para puro IPv6. El mecanismo (SPF, LSDB, L1/L2) es idéntico.
- **El plano de control se está moviendo a Segment Routing**: `RFC 8402` (2018), con **SR-MPLS (`RFC 8660`)** y **SRv6 (`RFC 8986`)**. Lo importante es el cambio de paradigma: **el estado pasa a estar solo en el borde de ingreso** y cada nodo aplica una política local, sin base de datos de caminos.
- **MPLS se apoya en un IGP por debajo**, típicamente **OSPF o IS-IS**. Si el datacenter usa EVPN-VXLAN, el plano de control es **BGP** (próxima clase) y **OSPF deja de usarse como IGP**. ⇒ **La consecuencia conceptual: OSPF no se usa para calcular rutas en datacenter porque el dominio es demasiado grande y demasiado plano.** La jerarquía que el IGP no te da, te la da BGP.
- **`BGP-LS` (Link-State) es hoy** `RFC 9552` (dic-2023, obsoleta `RFC 7752`): distribuye información de link-state **dentro de BGP** para que un controlador externo calcule. Es la manera moderna de sacar Dijkstra del router y ponerlo en un servidor.
- **Lo que NO se pregunta**: LSA **opacos**, *link-local signaling*, extensiones para *multi-topology*. Estandarizadas sí, pero la cátedra no las da.

Consecuencia para el examen: `2009-02 Problema 2a` y `2007-03 Problema 2` se resuelven con **el algoritmo y el flooding, que no cambiaron**. Si el enunciado menciona OSPF por nombre, contestá el mecanismo y **agregá al final una línea de actualización**: "*hoy el equivalente para IPv6 es OSPFv3 (RFC 5340) y el IGP link-state más desplegado en operadores es IS-IS*".

1.8 La elección del DR/BDR

En una red de acceso múltiple (*multiaccess*: Ethernet, FDDI, token ring), **todos los routers están en el mismo dominio de colisión lógico**. En una red punto a punto cada router tiene **exactamente un vecino**;

en una multiacceso, tiene **todos los demás**.

```
Red multiacceso con N routers (sin DR):
   2 routers →  1 vecindad      5 routers →  10 vecindades
   4 routers →  6 vecindades   20 routers → 190 vecindades
   con N routers:  vecindades = N(N-1)/2

┌────┐  ┌────┐  ┌────┐  ┌────┐  ┌────┐        Cada par (i,j) tiene que
│ R1 ├──┼──┤ R2 ├──┼──┤ R3 ├──┼──┤ R4 │        negociar una vecindad
│    │  │  │    │  │  │    │  │  │    │        OSPF completa: DBD, LSR,
└────┘  └──┼─┘    └──┼─┘    └──┼─┘    └──┘        LSU, LSAck, más el
           └─────────┴─────────┴──┘                refresco cada 30 minutos.

┌──────────────── RED MULTIACCESO ─────────────────┐
│   R1 ─┐  R2 ─┐  R3 ─┐  R4 ─┐  R5                │
│       └── DR ─┘      │      │                   │
│   vecindades FULL:                               │
│      DR  ⇔  R1, R2, R3, R4, R5     = 5           │
│      BDR ⇔  DR                      = 1          │
│      los demás entre sí: NO (se quedan en 2-Way) │
│   TOTAL = 6  en lugar de 10                      │
└──────────────────────────────────────────────────┘

SIN DR:     N(N-1)/2   =  N²/2
CON DR:     2N - 3      ≈  N

N = 5: 10 → 6 (40 %)   ·   N = 1000: 499.500 → 1997 (99,6 %)

La diferencia pasa de CUADRÁTICA a LINEAL. Eso es todo lo que
hace el DR: convertir un O(N²) en un O(N).
```

Y el LSA de tipo 2 es el precio que se paga. Como hay uno solo que habla por la red, es el DR quien genera el **Network-LSA (tipo 2)**: "en esta red multiacceso están estos `K` routers con estos Router IDs, y la red tiene este costo". Sin DR, **cada router** tendría que generar el suyo con la lista completa, y la base explotaría.

```
La elección, en detalle:
  1. ¿Mi Router ID es MAYOR que el del vecino en la interfaz?
       SÍ  → me postulo           NO  → él se postula
  2. ¿Tengo la MAYOR prioridad declarada en la interfaz?
       SÍ  → gano                 NO  → pierdo
  3. Prioridades iguales (1 en broadcast, 0 en p2p por omisión):
       gana el MAYOR Router ID.
El DR se re-elige si el DR actual cae; el BDR, entre los DROther.
En punto a punto NO hay DR: N va a 2.
```

1.9 La métrica: por qué "hops" puede ser pésima

Dijkstra asume que **hay una métrica** y que es **lo que se quiere minimizar**. Lo que no dice es *cuál*.

```
Una métrica de enrutamiento tiene que responder a UNA pregunta:
    ¿qué le importa al usuario de este camino?
"menos latencia"         → métrica = retardo
"más ancho de banda"     → métrica = ancho de banda (invertida)
"menos pérdida"          → métrica = tasa de pérdida
"estabilidad"            → métrica = preferencia de política
"que la ruta sea simple" → que la ruta sea la más SIMPLE

Y "cantidad de hops" no responde a ninguna de las cinco.
```

El caso que hace daño, y por qué es antiintuitivo. El caso "10 saltos contra 1 salto" **no** hace daño: si un camino son 10 saltos de 1 Mbit/s y el otro 1 salto de 100 Mbit/s, la métrica de hops elige el de menos saltos, o sea el rápido, y acierta. El caso que hace daño es el **inverso**:

```
┌──────────────────────────────────────────────────────────────┐
│  Camino A:   1 salto    ·  10 Mbit/s   ← lento pero corto    │
│  Camino B:  10 saltos   ·  1 Gbit/s    ← rápido pero largo   │
│                                                              │
│  ¿Cuál elige "hops"?  →  Camino A (1 < 10).                  │
│  ⚠ ¿Y cuál es el que el usuario quiere?  →  Camino B.        │
│                                                              │
│  Porque el cuello de botella no lo fija la cantidad de       │
│  saltos, lo fija EL ENLACE MÁS ESTRECHO DE CADA CAMINO.      │
└──────────────────────────────────────────────────────────────┘
```

Y la **métrica de OSPF** es justamente una respuesta a esto:

```
OSPF no usa "cantidad de saltos". Usa el COSTO DE INTERFAZ, que es
un entero de 16 bits derivado del ancho de banda:

     costo_interfaz =  ancho_de_banda_de_referencia
                       ──────────────────────────────
                          ancho_de_banda_de_la_interfaz

Con la referencia por omisión del RFC (100 Mbit/s):
     10 Mbit/s  → 100/10 = 10      100 Mbit/s → 100/100 = 1
     1 Mbit/s   → 100/1  = 100     2 Gbit/s   → 0,05 → 0 → piso a 1

⚠ Ese último renglón es el problema del campo de 16 bits: todo lo
más rápido que la referencia da costo 0. Por eso los equipos
modernos configuran la referencia en 100 Gbit/s. Es un parche
empírico sobre una decisión de diseño de 1998 que ya no escala.

Con referencia en 100 Gbit/s:
     10 Mbit/s → 10.000     1 Gbit/s → 100     100 Gbit/s → 1
     Ahora sí hay rango: el campo de 16 bits llega hasta 65.535.
```

> **Lo que OSPF NO tiene: no hay ancho de banda en la métrica, hay COSTO**, y el costo se configura a mano. Por eso el OSPF por sí solo no balancea carga: si no ponés el costo vos, elige el camino con menos saltos.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Dijkstra en 6 nodos, otro grafo

Datos. Seis nodos `a, b, c, d, e, f` con estos costos: `a–b` 4, `a–c` 1, `a–d` 2, `b–c` 7, `b–d` 2, `b–f` 5, `c–d` 5, `c–e` 9, `d–e` 6, `d–f` 3, `e–f` 4.

```
a ──4── b        a ──1── c        b ──2── d
a ──9── c        b ──7── c        c ──5── d
a ──2── d        c ──9── e        d ──6── e
b ──5── f        d ──3── f        e ──4── f
```

Paso 1 — Inicialización. Los vecinos de `a` son `b` (4), `c` (1) y `d` (2). Los otros, `∞`. Entra `c`.

Paso 2 — Relajo los vecinos de `c` fuera de `N'`: `b`, `d`, `e`.

```
b:  min( 4 , 1 + 7 ) = min(4, 8) = 4   → NO mejora
d:  min( 2 , 1 + 5 ) = min(2, 6) = 2   → NO mejora
e:  min( ∞ , 1 + 9 ) = min(∞,10) = 10  → SÍ, 10 vía c
```

Paso 3 — Entra `d` (`D(d)=2`). Vecinos de `d` fuera de `N'`: `b`, `e`, `f`.

```
b:  min( 4 , 2 + 2 ) = min(4, 4) = 4    → empate, se queda el original
e:  min( 10, 2 + 6 ) = min(10, 8) = 8   → SÍ, 8 vía d
f:  min( ∞ , 2 + 3 ) = min(∞, 5) = 5    → SÍ, 5 vía d
```

Paso 4 — Entra `b` (`D(b)=4`). Vecinos de `b` fuera de `N'`: solo `f`. `min(5, 4+5) = 5` → NO mejora.

Paso 5 — Entra `f` (`D(f)=5`). Vecinos de `f` fuera de `N'`: `e`. `min(8, 5+4) = 8` → NO mejora.

Paso 6 — Entra `e` (`D(e)=8`). No quedan nodos fuera de `N'`: fin.

```
Paso  N'           D(b)     D(c)     D(d)     D(e)      D(f)    nodo
──────────────────────────────────────────────────────────────────────
  0   {a}          4,a      1,a      2,a       ∞         ∞       –
  1   {a c}        4,a       –       2,a     10,c        ∞       c
  2   {a c d}      4,a       –         –       8,d       5,d      d
  3   {a c d b}     –        –         –       8,d       5,d      b
  4   {a c d b f}   –        –         –       8,d        –       f
  5   {a c d b f e} –        –         –        –         –       e

RESULTADO (caminos óptimos desde a):
  b : costo 4   a –4– b          e : costo 8   a –2– d –6– e
  c : costo 1   a –1– c          f : costo 5   a –2– d –3– f
  d : costo 2   a –2– d
```

> **Interpretación:** `e` a costo 8 **no** se encontró por la vía directa, sino relaxing `d`. El camino óptimo suele entrar por un nodo que no es el vecino directo.
>
> ⚠️ **Errores típicos: relajar nodos que ya están en `N'`** (rompe la invariante, es el error más caro); **actualizar todo el vector en cada paso** (solo los vecinos del nodo que acaba de entrar); **no escribir**

> **el predecesor** (sin él no hay árbol ni tabla de forwarding).

2.2 Ejercicio 2 — Las fases del arranque de OSPF

Enunciado. `R1` (Router ID `10.0.0.1`) y `R2` (`10.0.0.2`) por un enlace punto a punto. `R1` tiene en su LSDB `L1` (Router-LSA suyo), `L2` (Router-LSA de `R2`, viejo) y `L3` (Summary-LSA de `192.0.2.0/24`). `R2` tiene `L1`, `L2` y `L4` (Summary-LSA de `198.51.100.0/24`). Identificar qué mensaje se manda y en qué estado están las vecindades.

```
1. Hello (tipo 1)  a 224.0.0.5, TTL 1  --  R1 se presenta
2. Hello (tipo 1)  a 224.0.0.5, TTL 1  --  R2 contesta con su Router ID
3. DBD   (tipo 2)  unicast a R2         --  R1 describe su LSDB (3 headers)
4. DBD   (tipo 2)  unicast a R1         --  R2 describe el suyo (3 headers)
```

Ambos quedan en `Exchange`, y se cruzan dos DBD de descarte.

```
b) R1 ve que le falta L4:
R1 → R2 :  Link State Request (tipo 3): "necesito el LSA L4"
R2 → R1 :  Link State Update  (tipo 4): "acá va, más lo que te falte"
R1 → R2 :  Link State Acknowledgment (tipo 5): "recibido"

Ambos pasan Exchange → Loading → Full. Como a R1 le falta L4 y
a R2 le falta L3, el ciclo 3-4-5 se repite una vez por cada LSA.

c) SOLO el Hello es periódico: cada 10 s, indefinidamente. Si pasan
40 s sin Hello, el DeadTimer expira y la vecindad cae a Down.
Los otros cuatro son POR EVENTO: en el arranque, y cada 30 minutos.
Si los cinco fueran periódicos cada 10 s, una red con 200 routers y
3 vecindades gastaría ancho de banda todo el día para confirmar
algo que no cambió.

d) Si R2 deja de mandar Hello durante 50 s:
t = 0    llega el último Hello; el DeadTimer arranca en 40 s.
t = 40   expira en R1 → vecindad a Down, desarma el estado y lo
         anuncia por flooding como un Router-LSA con Age = MaxAge.
t = 40..50  R2 todavía no expiró el suyo: SIGUE INSTALANDO RUTAS POR
         R1 Y ENVIANDO PAQUETES. La caída es bilateral pero
         DESALINEADA. Por eso los timers son ajustables: en un
         enlace satelital un DeadInterval de 40 s es inservible.
```

2.3 Ejercicio 3 — Por qué OSPF no necesita split horizon

Enunciado. Explicá por qué *split horizon*, *poison reverse* y *hold-down*, **obligatorios** en distance-vector, **no existen** en OSPF, y con qué mecanismo se reemplazan.

Paso 1 — Qué es el split horizon. En DV, el conteo a infinito viene de que `R2` le cuenta a `R1` que llega a `Internet` a distancia 3 **usando a `R1` mismo como parte del camino**. Split horizon es: "*no le anuncies a* χ *una ruta cuyo próximo salto es* χ". Es un parche contra el rumor.

Paso 2 — En OSPF el problema ni siquiera puede plantearse.

```
En OSPF, el LSA de R2 dice:  "TENGO UN ENLACE HACIA R3 Y MI COSTO ES 5"

¿Dónde está el camino de R2 a R1?   NO ESTÁ.
¿Dónde está el camino de R2 a Z?    NO ESTÁ.

El LSA es un HECHO LOCAL, no una CONCLUSIÓN. El "llegar a Z con
costo 12" no lo anuncia nadie: lo CALCULA cada router por su
cuenta, localmente, con Dijkstra.

DISTANCE-VECTOR: la ruta es un DATO que viaja. Un router la puede
                  usar como parte de OTRA ruta que no conoce.
                  → rutas circulares → conteo a infinito
LINK-STATE:      los enlaces son DATOS que viajan. Las rutas son
                  CÁLCULOS locales que NUNCA viajan.
                  → no hay nada que pueda reintroducir un rumor
```

Paso 3 — Los controles que reemplazan a los de DV.

Problema de DV	Cómo lo resuelve OSPF
Un router se realimenta con su propio rumor	El rumor **no existe**: los LSAs son hechos locales, las rutas se calculan
Un LSA da vueltas en un anillo	**No reenvío por la interfaz de entrada** + **no reenvío si ya tengo esa instancia**
Un LSA viejo viaja tarde y pisa uno nuevo	**LS Sequence Number**: se compara el número, no el reloj de la red
Un LSA "vuelve" después de que la red se partió	**LS Age + MaxAge**: el origen lo tira explícitamente con un *flush*
Una ruta queda "colgada" al caerse un enlace	El **DeadInterval** detecta la caída en 40 s o menos, y el LSA del enlace desaparece del LSDB de todos

Paso 4 — La comparación final, que es lo que se escribe en el examen.

```
En DV, los controles existen porque el PROBLEMA está en la
información que viaja: es una estimación que puede estar vieja y
puede referirse a un camino que incluye al que la recibe.

En link-state, lo que viaja NO es una estimación sino un hecho. Y
los hechos no se realimentan, porque un hecho no "se copia de
vuelta": A inunda su enlace con B, y B ya tiene ese hecho desde
el momento en que lo leyó en el cable.

Por eso el costo de no necesitar split horizon se paga en otro
lado: memoria (LSDB completo) y CPU (Dijkstra entero).
Es un intercambio: estado por simplicidad de protocolo.
```

> ⚠️ **Errores típicos:** decir "*OSPF no tiene conteo a infinito porque es más rápido*". **La velocidad no tiene nada que ver.** Y no olvidar que **IGRP y EIGRP tampoco usan split horizon**: es una propiedad de la familia de algoritmos, no un accidente. Por último, **split horizon** (regla de contenido: "no le digas a nadie lo que te llega por él") es distinto de **no reenviar por la interfaz de entrada** (regla de topología). OSPF usa las dos, por razones distintas.

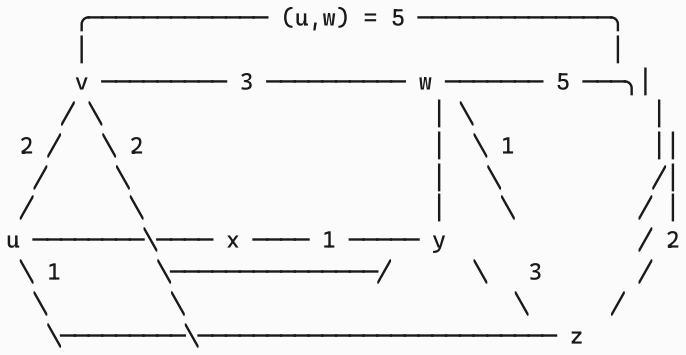
PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2009-02, Problema 2 (30 puntos) — Dijkstra desde `u`

> **Fuente:** Examen `eirc0902` / `sirc0902`, 2 de febrero de 2009. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0902.pdf **Texto del examen:**
>
> > El algoritmo de enrutamiento de Dijsktra es link state, donde cada nodo conoce la topología de la red y el costo de cada enlace. Se pide:
> >
> > a) Para la red de la figura, ejecute el algoritmo paso a paso en el nodo u hasta que logra establecer el camino más corto al resto de los nodos. Justifique detalladamente.
> >
> > b) Para el nodo u, de la tabla de forwarding, y dibuje el shortest-path tree resultante de la aplicación del algoritmo de Dijsktra.

La topología — *reconstruida desde las transparencias de la solución oficial*

La figura del enunciado no sobrevivió a la extracción de texto, así que la **reconstruí desde la solución oficial**: el PDF incluye, entre las páginas 9 y 10, dos transparencias del curso (`Int. Redes de Computadoras – Capa de Red`, hojas **4-76, 4-77 y 4-78**) que son la resolución oficial del problema. De ellas sale la red exacta:



Y, para que no quede ninguna ambigüedad, **el listado completo de los diez enlaces**:

Enlace	Costo		Enlace	Costo
`u – v`	2		`w – x`	3
`u – x`	1		`w – y`	1
`u – w`	5		`x – y`	1
`v – w`	3		`w – z`	5
`v – x`	2		`y – z`	2

> ⚠ **Discrepancia que hay que declarar.** La parte b) dice "*de la tabla de forwarding*" (con artículo definido), o sea que la figura debía incluir la tabla de routing de `u`. **Esa parte no se ve en el PDF**: la solución oficial solo publica la parte b) resuelta, como árbol y tabla. Si en el examen te dan una tabla de forwarding como dato de entrada, el procedimiento es idéntico: leés los costos y seguís los cinco pasos.

Parte a) — La tabla de iteraciones completa

Paso 0 — Inicialización. `N' = {u}`. Vecinos de `u`: `v` (2), `x` (1), `w` (5). Los demás, `∞`.

Paso	N'	D(v)	D(w)	D(x)	D(y)	D(z)	Entra
0	u	2,u	5,u	**1,u**	∞	∞	—

Paso 1 — Entra `x` (`D(x)=1`, el mínimo). Relajo los vecinos de `x` fuera de `N'`: `v`, `w`, `y`.

```
v:  min( 2 , 1 + 2 ) = min(2, 3) = 2  → NO
w:  min( 5 , 1 + 3 ) = min(5, 4) = 4  → SÍ, vía x
y:  min( ∞ , 1 + 1 ) = min(∞, 2) = 2  → SÍ, vía x
z:  x NO es adyacente a z. No se toca.
```

Paso	N'	D(v)	D(w)	D(x)	D(y)	D(z)	Entra
1	u x	2,u	4,x	—	**2,x**	∞	x

Paso 2 — Entra `y` (`D(y)=2`). Vecinos de `y` fuera de `N'`: `w` y `z`.

```
w:  min( 4 , 2 + 1 ) = min(4, 3) = 3  → SÍ, vía y
z:  min( ∞ , 2 + 2 ) = min(∞, 4) = 4  → SÍ, vía y
```

Paso	N'	D(v)	D(w)	D(x)	D(y)	D(z)	Entra
2	u x y	2,u	**3,y**	—	—	4,y	y

Paso 3 — Entra `v` (`D(v)=2`). Vecinos de `v` fuera de `N'`: solo `w`. `min(3, 2+3) = 3` → NO mejora.

Paso 4 — Entra `w` (`D(w)=3`). Vecinos de `w` fuera de `N'`: `z`. `min(4, 3+5) = 4` → NO mejora.

Paso 5 — Entra `z`. No quedan nodos fuera de `N'`: fin.

La tabla compacta (la que se copia en el examen)

```
Paso  N'          D(v),p   D(w),p   D(x),p   D(y),p   D(z),p    nodo
──────────────────────────────────────────────────────────────────────
  0   {u}         2,u      5,u      1,u       ∞        ∞         –
  1   {u x}       2,u      4,x       –       2,x       ∞         x
  2   {u x y}     2,u      3,y       –        –       4,y        y
  3   {u x y v}    –       3,y       –        –       4,y        v
  4   {u x y v w}  –        –        –        –       4,y        w
  5   {u x y v w z} –      –        –        –        –          z

Tabla oficial (solución del examen, hoja 4-77):
  Step  N'       D(v)  D(w)  D(x)  D(y)  D(z)
    0   u        2,u   5,u   1,u    ∞     ∞
    1   ux       2,u   4,x   –     2,x   ∞
    2   uxy      2,u   3,y   –     –     4,y
    3   uxyv     –     3,y   –     –     4,y
    4   uxyvw    –     –     –     –     4,y
    5   uxyvwz   –     –     –     –     –

COINCIDEN LAS CINCO ITERACIONES Y LOS CINCO VALORES DE DISTANCIA.
```

Interpretación (lo que hay que escribir después de la tabla):

```
D(v)=2 vía u → u → v            D(w)=3 vía y → u → x → y → w
D(x)=1 vía u → u → x            D(z)=4 vía y → u → x → y → z
D(y)=2 vía x → u → x → y

Fijate en D(w): el enlace DIRECTO u→w cuesta 5, pero llegar por
u→x→y→w cuesta 1+1+1 = 3. El camino más corto NO es el más directo,
y esa es la razón de ser del algoritmo. Un protocolo "más vecino
gana" habría elegido u→w con costo 5.
```

Parte b) — El shortest-path tree y la tabla de forwarding

```
u (raíz)
│
├── v                                  [ D(v)=2, p(v)=u ]
│
└── x                                  [ D(x)=1, p(x)=u ]
     │
     └── y                             [ D(y)=2, p(y)=x ]
          │
          ├── w                        [ D(w)=3, p(w)=y ]
          │
          └── z                        [ D(z)=4, p(z)=y ]

Verificación estructural: 6 nodos y 5 aristas, conexo → es un árbol.
Propiedad que se cumple SIEMPRE en el resultado de Dijkstra y que
sirve para autocontrolar.
```

Destino	Próximo salto	Costo	Interfaz de salida en `u`
`u` (local)	—	0	local
`v`	`v`	2	directo a `v`
`x`	`x`	1	directo a `x`
`y`	`x`	2	por `x`
`w`	`x`	3	por `x`
`z`	`x`	4	por `x`

Coincide con la tabla oficial del PDF (hoja 4-78), que da `destino → enlace` *como* `v → (u,v)`, `x → (u,x)`, `y → (u,x)`, `w → (u,x)`, `z → (u,x)`. Nótese que **la oficial no incluye la columna de costo**: para un link-state interno el costo es un detalle interno del SPF, y lo que importa para el forwarding es **por qué interfaz**. Es un detalle de diseño real: la FIB guarda **próximo salto e interfaz**, no costos.

Análisis de la solución oficial

✅ **Lo que está bien:** (1) la tabla tiene las cinco columnas de distancia más el `N'` acumulado, exactamente el formato que exige "paso a paso" + "justifique detalladamente"; (2) los nodos que entran a `N'` se ven marcados con flechas en la tabla original, y eso es lo que convierte una tabla en una *justificación*; (3) borra la `D` de los nodos ya definitivos, que es la convención del curso.

⚠️ **Lo que la oficial NO publica, y hay que resolver:** (1) **la parte b) está a medias**: el PDF pone el título y después la transparente con el árbol y la tabla, **sin una línea de texto**. La parte b) son **dos** entregables; si entregás solo la tabla, perdés la mitad. (2) **No hay justificación de las cuentas**: la tabla dice `D(w)=4,x` pero no dice por qué, y el enunciado pide "justifique detalladamente". Hay que escribir el `min` de cada relajación: la tabla **más las cinco ecuaciones** sí es la justificación. (3) **No hay verificación**: sumá el chequeo estructural (`|aristas| = |nodos| − 1 = 5`) y el aritmético (`D(z) = 1 + 1 + 2 = 4`).

Mi respuesta completa, en tres bloques:

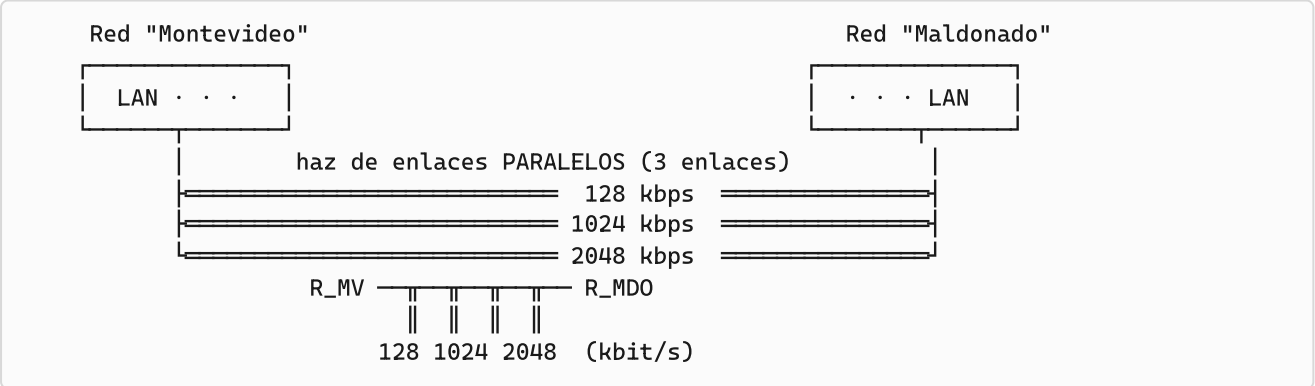
> **a)** Inicializo `N' = {u}`, `D(v)=2`, `D(x)=1`, `D(w)=5`, `D(y)=D(z)=∞`. En cada paso elijo el mínimo entre los nodos **no** definitivos, lo agrego a `N'` y relajo **solo** sus vecinos que están fuera de `N'` con `D(v) ← min(D(v), D(w) + c(w,v))`. Las cinco iteraciones dan `D(v)=2` por `u`, `D(x)=1` por `u`, `D(y)=2` por `x`, `D(w)=3` por `y`, `D(z)=4` por `y`. Verifico que el camino a `w` de costo 3 es más barato que el directo de costo 5.
>
> **b)** El árbol cuelga cada nodo del que lo mejoró: raíz `u`; de `u` cuelgan `v` y `x`; de `x` cuelga `y`; de `y` cuelgan `w` y `z`. La tabla de forwarding traduce el predecesor a interfaz: `v → directo`, `x → directo`, y `y`, `w`, `z` **por** `x`. Notar que cuatro destinos distintos salen por la misma interfaz, que es lo que hace que la FIB sea una tabla de prefijo corto en vez de una tabla por ruta completa.

3.2 ⭐ Examen 2007-03, Problema 2 (30 puntos) — enlaces paralelos y oscilación de métrica

> **Fuente:** Examen `eirc0703` / `sirc0703`, 26 de febrero de 2007. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0703.pdf **Texto del examen:**
>
> > Se desea analizar y mejorar el comportamiento de un algoritmo de enrutamiento link-state en una topología particular como la de la figura, en la que hay dos grupos de redes interconectadas por un haz de enlaces paralelos.
> >
> > a) Describa en términos generales el funcionamiento de un algoritmo de enrutamiento del tipo "link state" (OSPF). b) ¿Como estima que el mismo se comportará en una topología como la de la figura? ¿Que problemas podrían ocurrir en situaciones de alto tráfico y congestión? Estudiar en particular los casos de tráfico mayor a 2048 kbps y entre 1024 y 2048 kbps. c) Proponga y especifique en un lenguaje de alto nivel, un mecanismo para estimar el ancho de banda de los enlaces. Este mecanismo deberá ser lo menos intrusivo posible, por lo que se sugiere aprovechar el envío de LSAs (Link State Advertisements) para ello. Suponga que puede marcar el tiempo al momento de salida/entrada del paquete al enlace, y los routers tienen su tiempo exactamente sincronizado. d) ¿Cómo puede mejorar la situación de la parte (b) el conocer el ancho de banda de los enlaces, para la selección del mejor camino?

La topología — *reconstruida del PDF*

La figura es simple y el enunciado la describe bien: **dos grupos de redes interconectados por un haz de enlaces paralelos**. La del PDF tiene exactamente tres enlaces, de **128, 1024 y 2048 kbps**.



Lo que hay que entender antes de resolver nada: los tres enlaces tienen **el mismo costo por definición de la topología del ejercicio** (son enlaces entre el mismo par de nodos, así que OSPF los trata como tres enlaces más del grafo), pero **capacidades distintas**. Y el tráfico ofrecido de `Montevideo` a `Maldonado` es una sola cifra: si es 1500 kbit/s, tiene que ir **todo** por un enlace, porque Dijkstra devuelve **un solo camino**.

Parte a) — Funcionamiento general de link-state

Solución oficial (literal del PDF): cinco pasos (descubrir vecinos, medir distancias a vecinos, armar el paquete y mandarlo a todos, escuchar la topología y armar el grafo, calcular el camino más corto), más los comentarios: los pasos 1 y 2 usan **HELLO** que "solamente tienen significado local entre vecinos y **no deben ser enrutados**"; el paso 3 arma un **LSA**; el paso 5 corre **Dijkstra**; y para asegurar que los LSA lleguen a todos se **inundan (flooding)** "con mecanismos para descartar los LSA ya vistos y números de secuencia para garantizar que todos los routers tienen información consistente de la topología". Y una frase final que vale puntos: "*La métrica puede ser cualquier cosa que se pueda medir. Desde número de saltos pasando por ancho de banda y retardo*".

Qué le agregaría, y es lo que suma puntos:

Punto	Por qué suma
La LSDB es idéntica en todos los routers del área	Es la condición de **sincronización**; sin ella el protocolo no sirve
El cálculo de Dijkstra es local, no global	Es lo que hace escalable el esquema y la diferencia con un router centralizado
"Los HELLO no deben ser enrutados" se implementa con `TTL = 1`	Convierte "no deben" en un mecanismo
Las áreas, con el cálculo de O(N) en estado	Justifica por qué existe el OSPF de verdad
La métrica de OSPF es costo de interfaz, derivado del ancho de banda	Conecta directo con las partes b) y c)

Parte b) — El comportamiento con tráfico alto: la oscilación

Solución oficial (literal): "*El algoritmo tal como fue descripto en el teórico corre el algoritmo de Dijkstra simple, que para cada destino va a devolver solamente el camino más corto en cada momento de corrida. Por ello la red en cada momento que se corra el D. va a elegir uno de los enlaces.*" … "*Mientras el tráfico ofrecido es < que 2048 kbps, el enlace elegido es el de 2048 kbps.*" … "*A medida que el tráfico ofrecido se acerca a 2048 la métrica sobre este enlace empeora mientras que la métrica por el de 1024 se mantiene constante (pensar en que si mando HELLOs los mismos tienen que esperar cada vez más en cola para ser transmitidos).*" … "*A medida que nos introduce en congestión más profunda, hay un momento de corte en el cual la métrica por el enlace de 1024 es mejor que por el saturado enlace de 2048 y el enlace elegido pasa a ser el de 1024.*" … "*¿PERO qué pasa? El tráfico ofrecido es mucho mayor que 1024 por lo que este nuevo enlace satura casi inmediatamente y un breve tiempo después el enlace de 2048 vuelve a ser el mejor.*" … "*Este ciclo de oscilaciones se puede repetir varias veces mientras la carga ofrecida no varíe. Esta situación es muy perniciosa para la red y ocasiona importantes pérdidas de paquetes.*"

El mecanismo, gráfico con números. La clave está en esa frase entre paréntesis: **la métrica no es estática, cambia con el tráfico**. Supongamos `costo = ancho_de_banda_referencia / ancho_de_banda_efectiva`, con referencia 2048:

```
Ofrecido: 1500 kbit/s   (el MISMO todo el tiempo)

t = 0   2048 al 80 % → efectivo 1638 kbit/s → costo = 2048/1638 = 1,25
       1024 vacío    → efectivo 1024        → costo = 2048/1024 = 2,00
        128 vacío    → efectivo  128        → costo = 2048/128  = 16,0
       → elige el de 2048 (1,25).  ✓ CORRECTO: cabe 1500 < 1638

t = 1   la cola del 2048 crece:
       2048 al 95 % → efectivo 973 → costo = 2048/973 = 2,10
       1024 al  0 % → costo = 2,00
       ⚠ AHORA el de 1024 es MÁS BARATO (2,00 < 2,10) y Dijkstra
         cambia el camino... aunque 1500 > 1024, o sea que el
         enlace de 1024 NO ALCANZA para todo el tráfico.

t = 2   los 1500 kbit/s llegan al 1024. Se satura en segundos:
       1024 al 100 % → efectivo 350 → costo = 2048/350 = 5,85
       → el de 2048 (que sigue al 95 %) vuelve a ser el más barato.

t = 3   el 2048 también satura → costo = 16,0 ; el 1024 sigue en
       5,85 → ahora gana el 1024 otra vez. Y vuelta a empezar.

RESUMEN DE LA OSCILACIÓN, en una línea de tiempo:

  t0 ── el 2048 gana ──┬── t1 el 1024 "gana" ──┬── t2 el 2048
                       │                       │
  (carga 1500, que     └── t3 el 1024 ─────────┘   ... y vuelta
   NUNCA entra entero
   en el de 1024)
```

El nombre del problema: metric-driven oscillation, u oscilación de métrica; en la literatura de operación de redes, **"flapping"**. Es un caso particular de **inestabilidad de la decisión de enrutamiento**: ocurre cuando la realimentación de la métrica es más rápida que la constante de tiempo del tráfico.

Por qué es tan dañina: (1) **descarte**, al pasar 1500 kbit/s de golpe a un enlace de 1024 los 478 kbit/s que no entran se pierden; (2) **latencia**, porque durante la oscilación cada paquete va por un camino distinto y los retardos se disparan; (3) **sesiones TCP**, que cambian de camino a mitad de vida, con lo que eso implica para la retransmisión y el RTO; (4) **CPU**, porque cada cambio recalcula SPF y genera flooded, y con oscilación el flooded es continuo.

Y el caso particular que pide el enunciado, que es un detalle fino y vale puntos. Pide estudiar por separado "*tráfico mayor a 2048 kbps*" y "*entre 1024 y 2048 kbps*" porque **el comportamiento cualitativo cambia**:

Rango de tráfico ofrecido	Qué ocurre
< 1024 kbit/s	**Zona segura.** El enlace de 2048 alcanza siempre, nunca se satura, la métrica es estable.
1024 ≤ X < 2048 kbit/s	**Zona de oscilación pura, el caso peor.** Ningún enlace alcanza para todo el tráfico: 2048 alcanza y 1024 no. El 2048 se congestiona un poco y el 1024 "parece" mejor; al mover el tráfico, el 1024 se satura y vuelve el 2048. Oscilación de período corto.
X ≥ 2048 kbit/s	**Zona de saturación.** Los tres enlaces al 100 %. La métrica de los tres es mala, pero **el tráfico no cabe en ninguno**. La oscilación puede cesar (todo está igual de malo), pero **la red está colapsada**. El problema dejó de ser de enrutamiento y pasó a ser de **capacidad**.

Conclusión que hay que escribir: para tráfico entre 1024 y 2048 el problema **es de enrutamiento** (se arregla eligiendo bien el enlace); para tráfico mayor a 2048 es **de dimensionamiento** (no lo arregla ningún algoritmo). El enunciario pide que distingas las dos cosas porque **son soluciones distintas**.

Parte c) — Medir el ancho de banda aprovechando los LSA

Solución oficial (literal del PDF), cuatro pasos:

> 1. Armar los LSAs de un tamaño conocido y fijo, eventualmente con padding (relleno) para hacerlos de un largo no muy corto ni demasiado largo, por ejemplo, 1024 bytes. Esto es para eliminar efectos de borde de paquetes muy cortos o muy largos.
> 2. Enviar el LSA marcando el timestamp de salida, posiblemente en un campo de opciones adicionales.
> 3. El router que recibe el LSA con estas características determina el ancho de banda realizando una simple operación: `Bw = L / (Ts2 - Ts1)`, donde `Ts1` y `Ts2` son los timestamps (salida y llegada) y `L` es el largo del LSA.
>
> **Observación 1:** *La hipótesis de sincronización de todos los routers es fundamental, sin ella este proceso no es posible y hay que recurrir a otro tipo de técnicas mucho más complejas.* **Observación 2:** *La medida debería repetirse en varias oportunidades y promediar, a los efectos de eliminar efectos interferentes del resto del tráfico.*

Análisis de la oficial. Los cuatro pasos son correctos y la fórmula es la correcta. Pero se queda en medir el **tiempo de tránsito**, que es un *throughput* instantáneo, no un **ancho de banda** nominal:

```
⚠ LO QUE LA FÓRMULA MIDE ES throughput, NO ancho de banda.

Bw_aparente =  L / (T_salida - T_llegada)

 El tiempo (Ts2 - Ts1) incluye:
     · el tiempo de TRANSMISIÓN   (L / capacidad)   ← lo que querés medir
     · el tiempo de ESPERA EN COLA                  ← lo que NO querés
     · el tiempo de PROPAGACIÓN                    ← es constante, no importa

 En un enlace libre:   T ≈ L/cap + d_prop        → Bw medido ≈ capacidad  ✓
 En un enlace cargado: T ≈ L/cap + d_cola + d_prop → Bw medido MUCHO MENOR ✗

 O sea: con un enlace saturado, la fórmula mide menos ancho de banda
 del que tiene. ¿Y eso está mal? NO: para el problema del examen
 ESTO ES LO QUE QUEREMOS. Un enlace que rinde poco porque tiene
 cola es un enlace que no sirve para transportar tráfico.
```

Mi implementación en pseudocode C, con las dos observaciones implementadas:

```
/* Medición intrusiva del ancho de banda efectiva de un enlace,
   reutilizando el LSA de OSPF como paquete de prueba.
   La salida se marca en el router ORIGEN (una sola marca) y la
   llegada en el RECEPTOR. Nunca dos marcas.                      */

#define TAMANO_LSA     1024     /* bytes, tamaño fijo conocido        */
#define MAX_MUESTRAS   8        /* para promediar                      */
#define MAX_ESPERA     5        /* EPR de Kolmogorov: trunca el último */
                                  /* 5 % de las muestras (las del buffer */
                                  /* lleno, que son las que interesa)   */

typedef struct {
    int64 t_salida;    /* reloj del origen al encolar el LSA          */
    int64 t_llegada;   /* reloj del receptor al leer el último byte   */
    int   longitud;    /* bytes, siempre TAMANO_LSA                   */
} muestra_t;

/* La LSA es un paquete normal de OSPF: le metemos un TLV opcional.
   RFC 2328 admite options en el header del LSA.                      */

typedef struct {
    int    tipo_lsa;
    int    router_id;
    uint32 seq;           /* marca única, para no confundir con refresco */
    int64  marca_temporal;/* timestamp del router ORIGEN (microsegundos) */
    int    relleno;       /* padding hasta TAMANO_LSA, con patrón       */
                          /* conocido para verificar que no se truncó   */
} lsa_medicion_t;

/* ---- en el router ORIGEN, al emitir un LSA periódicamente ---- */

lsa_medicion_t construir_lsa_medicion(void)
{
    lsa_medicion_t m;
    m.tipo_lsa       = 1;          /* Router-LSA: el que ya inunda    */
                                   /* cada 30 min y en cada cambio    */
    m.router_id      = mi_router_id;
    m.seq            = ++contador_seq;
    m.marca_temporal = AHORA();    /* EL PUNTO DE LA MEDICIÓN         */
    for (int i = 0; i < TAMANO_LSA - sizeof(m); i++)
        ((uint8_t *)&m)[sizeof(m) + i] = (uint8_t)(i & 0xFF);
    return m;
}

/* ---- en el router RECEPTOR, al recibir un LSA con TLV ---- */

void procesar_lsa_medicion(lsa_medicion_t *m)
{
    /* 1. ¿Es una muestra válida? -- descartamos refrescos y corruptos */
    if (!tiene_tlv_medicion(m))     return;  /* LSA normal              */
    if (m->relleno_incorrecto())    return;  /* truncado o mal rellenado*/
    if (edad(m) > MAX_AGE)          return;  /* llegó tarde: el reloj    */
                                           /* ya no dice nada          */
    /* 2. Guardar la muestra */
    buffer[ptr].t_salida  = m->marca_temporal;
    buffer[ptr].t_llegada = AHORA();
    buffer[ptr].longitud  = TAMANO_LSA;
    ptr = (ptr + 1) % MAX_MUESTRAS;
    /* 3. Recalcular si ya hay muestras */
    if (ptr != 0) actualizar_metrica_del_enlace();
}
```

```
/* ---- estimación ------------------------------------------------ */

void actualizar_metrica_del_enlace(void)
{
    /* 3a. Throughput instantáneo de cada muestra */
    int64 tasa[MAX_MUESTRAS];
    for (int i = 0; i < MAX_MUESTRAS; i++) {
        double dt = (buffer[i].t_llegada - buffer[i].t_salida) / 1e6;
        if (dt <= 0.0) return;   /* relojes incoherentes: el sincronismo
                                    se rompió. NO promediar.            */
        tasa[i] = (int64)(buffer[i].longitud * 8 / dt);   /* bits/s     */
    }

    /* 3b. Truncar la cola superior. Las muestras más altas son
           las de mayor tasa medida, y son las que menos nos
           cuando el buffer está vacío el enlace rinde su capacidad
           real, que ya la conocemos por configuración.                */
    ordenar_descendente(tasa, MAX_MUESTRAS);
    int utiles = MAX_MUESTRAS * (100 - MAX_ESPERA) / 100;
    if (utiles < 1) utiles = 1;

    /* 3c. Promediar el resto (el enunciado dice "promediar") */
    int64 suma = 0;
    for (int i = 0; i < utiles; i++) suma += tasa[i];
    capacidad_estimada = suma / utiles;

    /* 3d. Nunca por encima de la capacidad física */
    if (capacidad_estimada > capacidad_configurada)
        capacidad_estimada = capacidad_configurada;

    /* 3e. Confianza. Si las muestras son dispares, la estimación es
           mala y NO hay que cambiar la métrica: una muestra ruidosa
           provocaría una oscilación, que es EXACTAMENTE el problema
           que estamos tratando de arreglar.                         */
    varianza = desviacion(tasa, utiles, capacidad_estimada);
    confianza = (varianza < UMBRAL_VARIANZA) ? ALTA : BAJA;

    /* 3f. Solo se publica si hay confianza y si el cambio supera un
           margen de HISTÉRESIS, que es el antídoto contra la
           oscilación.                                                */
    if (confianza == BAJA) return;
    if (cambio_relativo(capacidad_estimada, costo_actual()) < MARGEN) return;
    costo_nuevo = COSTO_REFERENCIA * 1000 / capacidad_estimada;
    publicar_lsa_con_nuevo_costo(costo_nuevo);
}
```

La respuesta a "Observación 1" — la sincronización de relojes — merece desarrollo propio:

```
El enunciado dice "los routers tienen su tiempo exactamente
sincronizado". Eso es una HIPÓTESIS, y hay que decir qué se rompe
si es falsa:

· Si el reloj del receptor va ADELANTADO, (T2 − T1) es MENOR que la
  realidad y la tasa medida sale MÁS ALTA de lo que es.
· Si va ATRASADO, sale MÁS BAJA.
· Y como el reloj deriva, la deriva se acumula: un offset fijo de
  500 µs sobre un tiempo de tránsito de 300 µs es un error del 160 %.

Sin sincronización NO se puede medir así. Las alternativas son:
  · Ping-pong de challenge-response con el eco del timestamp: el
    receptor devuelve el timestamp recibido y se mide el tiempo de
    ida y vuelta; el error de offset se cancela. Es lo que hace
    cualquier estimación de RTT, incluido Jacobson/Karels en TCP
    (Clase 19). Es la respuesta correcta si el enunciado retira la
    hipótesis de sincronización.
  · NTP alcanza para el protocolo, pero no da precisión de
    microsegundos; no sirve para esto.
  · TWAMP es la herramienta de la UIT-T para medir ancho de banda
    sin carga intrusiva. Es el estándar industrial y no es un RFC
    de IETF: nombralo como "la forma industrial de hacer esto".
```

La respuesta a "Observación 2" — promediar — y por qué promediar NO alcanza:

Promediar reduce el ruido, pero **no arregla el sesgo sistemático** del buffer lleno: si el enlace está congestionado la mitad del tiempo, el promedio da un número que no describe ni el estado bueno ni el malo. Por eso la implementación agrega dos cosas que la oficial no pide pero que **son las que hacen que el método funcione**:

1. **Truncamiento de la cola superior** (`MAX_ESPERA`), no promedio simple: se descartan las muestras buenas y se promedian las malas, que son las que describen la contención.
2. **Histéresis**: la métrica solo se publica si el cambio supera un margen. **Sin histéresis, la solución a b) reproduce b).** Es el error clásico de realimentación positiva, y es el mismo error conceptual del AQM tardío en TCP (Clase 21): publicar la métrica de inmediato cuando cambia es lo que genera la oscilación.

Parte d) — Cómo mejora conocer el ancho de banda

Solución oficial (literal): "*En el caso más elemental, conocer el ancho de banda me permitiría simplemente evitar oscilaciones porque puedo saber a-priori que el nuevo enlace no va a soportar la carga. Conocido el ancho de banda en este escenario es posible utilizar versiones refinadas del algoritmo de Dijkstra que no solo determinan el mejor camino sino eventualmente también caminos alternativos.*"

```
SIN conocer el ancho de banda: la métrica es una estimación
ruidosa y el enrutamiento reacciona al ruido. Resultado:
oscilación 2048 ⇔ 1024.

CONOCIDO el ancho de banda (2048 / 1024 / 128 kbps), tres usos:

(1) MÉTRICA FIJA POR CAPACIDAD → elimina la oscilación
     costo = 2048 / capacidad
     2048 kbps → 1 ; 1024 kbps → 2 ; 128 kbps → 16
     Con oferta de 1500 kbps, gana siempre el de 2048, porque los
     otros NUNCA pueden con ese tráfico. Fin de la oscilación.

(2) MÉTRICA DE ANCHO DE BANDA RESIDUAL → reparto de carga
     bw_residual = capacidad − tráfico_ofrecido_por_este_enlace
     Con dos caminos de 2048: uno lleva 1000 y otro 500.
     bw_residual 1048 vs 1548 → el tráfico va al segundo.
     Esto es ECMP o el "link-state con ancho de banda", y es lo que
     se conoce como traffic engineering.

(3) DIJKSTRA MODIFICADO CON SEGUNDOS CAMINOS → redundancia
     En vez de UN camino por destino, se calculan los K mejores
     caminos disjuntos. Si el primario se cae, entra el segundo sin
     recalcular nada, y si el primario se degrada (y no cae), se
     reparte entre ambos. Es lo que el enunciario llama
     "versiones refinadas del algoritmo de Dijkstra".

El precio de las tres: (1) y (2) necesitan que la capacidad sea
conocida y estable; (3) multiplica por K el trabajo del SPF y
necesita memoria para K tablas. Y ninguna mide la CONGESTIÓN real:
por eso el esquema real es "capacidad conocida + ajuste fino por
cola", que es lo que hacen las Extensiones de OSPF para
Link-Bandwidth y las métricas ajustables de los fabricantes.
```

3.3 Examen 2009-12, Pregunta 3 (8 puntos) — el cometido de la capa de red y si "hops" es buena métrica

> **Fuente:** Examen `eirc0912` / `sirc0912`, 18 de diciembre de 2009. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0912.pdf **Texto del examen:**
>
> > a) ¿Cuál es el principal cometido de la capa de red? b) La longitud en "hops" es la métrica utilizada habitualmente para definir el costo de un enlace. ¿Es la más conveniente? Justifique su respuesta.

Solución oficial (literal del PDF):

> a) Asegurar la entrega de información de extremo a extremo. b) Si se busca una utilización eficiente de los recursos y/o satisfacer requerimientos de calidad de servicio, la métrica de hops puede ser inconveniente, ya que señalará como "mejor" el camino más corto, que no necesariamente será el

> que tenga mejores atributos de calidad (retardo extremo a extremo, pérdidas, throughput) y/o menos congestión.

Análisis de la solución oficial

La parte a) es corta pero incompleta, y hay que saber por qué. "*Asegurar la entrega de información de extremo a extremo*" es el cometido general de la capa de red, y es correcto como respuesta de 2-3 puntos. Pero la palabra **"asegurar"** es engañosa: en el curso *reliable* significa "que no se pierde nada", y eso **la capa de red NO lo garantiza**. Lo que sí garantiza es el **encaminamiento** a través de una internet de redes heterogéneas, con **decisión local** en cada nodo y sin necesidad de conocer la topología global. El servicio de IP es **best effort** (Clase 21 §1.6): lo que aporta la capa de red es direccionamiento jerárquico, *forwarding* por *longest prefix match* y enrutamiento.

Parte b) — El ejemplo con números

La oficial es correcta pero abstracta (nombra "retardo, pérdidas, throughput" sin ninguno). El enunciado pide **"justifique"**, y una justificación sin números es una afirmación.

```
CAMINO A:  10 saltos  ·  cada enlace de 10 Mbit/s  ·  1 ms de propagación
CAMINO B:   1 salto   ·  ese enlace de 1 Gbit/s    ·  0,5 ms de propagación

┌──────────────────────────────────────────────────────────────┐
│                    camino A      camino B                  │
│  hops                    10            1                   │
│  cuello de botella  10 Mbit/s   1000 Mbit/s  ← el más estrecho│
│  ¿cuál elige "hops"?         A  ✗  (1 < 10)                  │
│  ¿qué ancho de banda usable? 10 Mb/s        1000 Mb/s        │
│  ¿cuál es 100 veces mejor?  ──────────────→ B                │
└──────────────────────────────────────────────────────────────┘
```

El cálculo completo. Paquete de 1500 bytes = 12 000 bits. `d_proc = 0`.

```
── CAMINO A (10 saltos, 10 Mbit/s, 1 ms de propagación c/u) ──

d_trans por enlace = L / R = 12 000 / (10 × 10⁶) = 1,2 × 10⁻³ s = 1,2 ms

10 enlaces  →  10 × 1,2 ms  =  12,0 ms   (transmisión total)
10 enlaces  →  10 × 1 ms    =  10,0 ms   (propagación total)
procesamiento: 10 × 20 µs    =   0,2 ms
cola: 0 (sin congestión)
                               ─────────
RETARDO TOTAL camino A         =  22,2 ms

── CAMINO B (1 salto, 1 Gbit/s, 0,5 ms de propagación) ──

d_trans = 12 000 / 10⁹ = 1,2 × 10⁻⁵ s = 0,012 ms = 12 µs

1 enlace   →  1 × 0,012 ms =  0,012 ms   (transmisión)
1 enlace   →  1 × 0,5 ms   =  0,5 ms      (propagación)
procesamiento: 1 × 20 µs    =  0,02 ms
                               ────────
RETARDO TOTAL camino B         =   0,53 ms

Retardo:     22,2 / 0,53  =  42 veces mejor el camino B
Throughput:  1000 / 10    = 100 veces más ancho el camino B
```

Y para rematar el argumento de la oficial — pérdidas y throughput medido, no teórico:

```
El ancho de banda TEÓRICO de un camino no es su ancho de banda REAL.
El real es el del cuello de botella, reducido por slow start y por el
control de congestión de TCP:

camino A: 10 Mbit/s × 0,7 = 7 Mbit/s      camino B: 1 Gbit/s × 0,7 = 700 Mbit/s

Para una descarga de 100 Mbit/s:
camino A: NO PUEDE. Va a 7 Mbit/s, o sea el 7,3 % de lo pedido.
camino B: puede, y le sobra.

Y las pérdidas: el camino A tiene 10 enlaces, y la probabilidad de
que UNO de ellos pierda un paquete en una ventana es 10 veces mayor
que en el camino B. La probabilidad de que una sesión TCP sufra una
pérdida es proporcional al número de saltos.
```

La respuesta completa, como se escribe en el examen:

> **a)** El principal cometido de la capa de red es **encaminar los paquetes de un sistema final a otro a través de una internet de redes heterogéneas**, tomando una **decisión local** en cada nodo. La consequence es que **no necesita conocer la topología global** (de ahí los algoritmos escalables link-state y path-vector), y que **no garantiza fiabilidad**: el servicio de IP es *best effort*. Lo que aporta es direccionamiento jerárquico, *forwarding* por *longest prefix match* y enrutamiento por intercambio de información de control. La frase "asegurar la entrega" es engañosa: lo que se asegura es que el paquete llegue *a destino*, no que llegue *completo y en orden*.
>
> **b)** No, "hops" **no es la métrica más conveniente**, por tres razones concretas. **(1) No correlaciona con el retardo**: la transmisión es `L/R` y la propagación es `d/s`, y ninguno depende de la cantidad de saltos. **(2) No correlaciona con el ancho de banda**: el camino más corto en hops puede tener un cuello de botella de 10 Mbit/s frente a otro de 1 Gbit/s, como en el ejemplo numérico. **(3) No**

> **correlaciona con la pérdida**: más saltos son más puntos de falla y más transiciones de capa 2 que rehacer. Lo único con lo que "hops" correlaciona es el **retardo de procesamiento**, del orden de los microsegundos, **diez mil veces menor** que las diferencias de ancho de banda y propagación, y por lo tanto irrelevante. Lo que corresponde es una métrica compuesta (el **costo de interfaz** de OSPF, o ancho de banda más retardo en EIGRP) que refleje lo que el usuario necesita, y en general **configurable por el administrador**.

3.4 Examen 2007-12, Pregunta 1 (5 puntos) — los cinco mensajes de OSPF

> **Fuente:** Examen `eirc0712` / `sirc0712`, 27 de diciembre de 2007. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0712.pdf **Texto del examen:**
>
> > Explique resumidamente los 5 mensajes que se implementa en OSPF.
>
> ⚠ **Nota sobre la solución oficial:** el PDF de `sirc0712` publica la solución de los **Problemas prácticos** pero **no** la de las Preguntas teóricas. Lo que sigue es *(solución propia, contrastada con `RFC 2328`)*. La tabla de §1.7 es la respuesta y no hay que volver a inventarla.

#	Mensaje	Código	Para qué, en una línea
1	**Hello**	1	Descubre y mantiene vivos a los vecinos (cada 10 s, TTL 1, a `224.0.0.5` / `224.0.0.6`)
2	**Database Description**	2	En el arranque, cada uno **describe** lo que tiene en su LSDB para que el otro diga qué le falta
3	**Link State Request**	3	**Pide** los LSAs concretos que le faltan
4	**Link State Update**	4	**Manda** los LSAs pedidos o los nuevos por flooding
5	**Link State Acknowledgment**	5	**Confirma** la recepción

El razonamiento que hay que agregar, que es lo que diferencia un 4 de un 5:

1. **El Hello es el único periódico.** Los otros cuatro son del arranque y del refresco de 30 min. El Hello es el único que **no lleva información de la topología**: su contenido útil es "estoy vivo, mi Router ID es X, mi DeadInterval es Y".
2. **El Hello tiene TTL 1 y va a multicast**, los otros cuatro tienen TTL 1 y van a **unicast** al vecino concreto: el Hello tiene sentido entre todos los routers de la red, los otros solo entre dos.
3. **Los tres últimos forman un lazo de flooded confiable** (3 → 4 → 5) que existe **para no inundar a ciegas**: el `LS Update` sin acuse se reenvía a los 5 s, y el `LS Request` es la excepción que reemplaza al flooding masivo.
4. **El 2 es un "índice" y el 4 es el "contenido"**: el Database Description **no lleva las LSAs**, solo sus **headers** (LS ID, tipo, originator, número de secuencia, longitud). Por eso se llama *Description*. El que

las lleva enteras es el LS Update.

5. **El Hello cumple además tres funciones extra** que se preguntan: negociar los timers, **elegir el DR/BDR** (comparando prioridades y Router IDs) y detectar al vecino caído (`DeadInterval` = 40 s = 4 × `HelloInterval`).

La respuesta de 5 puntos, tal cual la escribirías:

> Los cinco mensajes de OSPF son: **Hello**, que se envía cada 10 segundos con TTL 1 a los multicast `224.0.0.5` y `224.0.0.6` y sirve para descubrir y mantener vivos a los vecinos, negociar los intervalos y elegir el DR/BDR, y cuya ausencia por 40 segundos declara la vecindad caída; **Database Description**, que en el arranque describe el índice del LSDB propio (los headers de sus LSAs, no las LSAs) para que el vecino determine qué le falta; **Link State Request**, con el que el vecino pide explícitamente las LSAs que le faltan; **Link State Update**, que es el único que transporta LSAs completas y sirve tanto para responder al request como para el flooding de cambios; y **Link State Acknowledgment**, que confirma la recepción de un Update. Los tres últimos forman el mecanismo de **flooding confiable**: sin acuse de recibo, el Update se reenvía a los 5 segundos.

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. **Link-state = cada router conoce la topología completa del área y corre el mismo algoritmo.** Distance-vector = cada router solo conoce lo que le dijeron sus vecinos.
2. La **LSA** describe **un enlace** con su costo; es un **hecho local**, no una estimación de ruta. El **LSDB** es idéntico en todos los routers del área, y por eso todos calculan el mismo árbol.
3. El **flooding** lleva cada LSA a todos. Se corta con tres controles: **no reenviar por la interfaz de entrada**, **no reenviar si ya tengo esa instancia**, y comparar por **número de secuencia**.
4. La **edad (MaxAge = 3600 s)** es el reloj que expira los LSAs; el **número de secuencia** con signo de 32 bits es el orden total que decide cuál es "nuevo".
5. **Dijkstra** mantiene la invariante "los nodos en `N'` tienen su distancia óptima y no va a cambiar", y la mantiene porque solo relaja vecinos **fuera** de `N'`. Por eso exige costos **no negativos**.
6. La tabla de iteraciones tiene **tres bloques**: `N'` (definitivos), `D(n), p` (distancia y predecesor) y el **nodo que entra en cada paso**. Sin ella no hay puntos.
7. **Complejidad**: Bellman-Ford es $O(V^3)$ con matriz; Dijkstra es $O(V^2)$ con matriz y $O(E \cdot \log V)$ con cola de prioridad. El **estado** es $O(N)$ por router y el flooding es $O(N)$ por evento: por eso existen las **áreas**.
8. Los **cinco mensajes** de OSPF son Hello (10 s, TTL 1, multicast), DBD (índice del LSDB), LSR (pedir), LSU (mandar/inundar) y LSAck (confirmar).
9. Las **áreas** segmentan el grafo; los **LSA de tipo 3** resumen otra área y los de **tipo 4** dicen cómo llegar al ASBR. El **DR/BDR** convierte $N(N-1)/2$ vecindades en $2N - 3$.
10. **"Hops" no es una buena métrica**: 10 saltos de 10 Mbit/s dan 22,2 ms y 1 salto de 1 Gbit/s da 0,53 ms. OSPF usa **costo de interfaz** derivado del ancho de banda.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Cuál es la diferencia estructural entre un LSA y un vector de distancias?

<details>
<summary>Respuesta</summary>
</details>

2. ¿Por qué Dijkstra necesita costos no negativos y Bellman-Ford no?

<details>
<summary>Respuesta</summary>
</details>

3. ¿Por qué la línea 12 del algoritmo dice "para todos los `v` adyacentes a `w` **y no en** `N'`"? ¿Qué se rompe si se saca esa condición?

<details>
<summary>Respuesta</summary>
</details>

4. Nombres los tres controles que impiden que el flooding de un LSA se.loop infinitamente, y qué pasaría si sacara cada uno.

▶ Respuesta

5. ¿Por qué la elección del DR/BDR es necesaria en un medio multiacceso y no en punto a punto?

▶ Respuesta

6. Explique por qué "hops" es una mala métrica, con las cuatro componentes del retardo.

▶ Respuesta

7. En el `2007-03`, ¿por qué medir el ancho de banda con timestamps en los LSA **no sirve** por sí solo contra la oscilación, y qué falta agregarle?

▶ Respuesta

8. OSPF no usa split horizon. ¿Por qué puede prescindir de él, y qué paga a cambio?

▶ Respuesta

🔗 Referencias

- **Examen `eirc0902` / `sirc0902`** (2-feb-2009) — Problema 2 completo (30 pts). La figura del enunciado no está en la extracción de texto; la red se **reconstruyó desde las transparentes de la solución oficial** (hojas 4-76, 4-77 y 4-78 del PDF), que incluyen la tabla de iteraciones.
- **Examen `eirc0703` / `sirc0703`** (26-feb-2007) — Problema 2 completo (30 pts): enlaces paralelos de 128/1024/2048 kbps, oscilación de métrica, medición de ancho de banda con LSA y timestamps.
- **Examen `eirc0912` / `sirc0912`** (18-dic-2009) — Pregunta 3 (8 pts): el cometido de la capa de red y si "hops" es la métrica más conveniente.
- **Examen `eirc0712` / `sirc0712`** (27-dic-2007) — Pregunta 1 (5 pts): los cinco mensajes de OSPF. ⚠ El PDF **no publica la solución teórica**; la resolución de §3.4 es *(solución propia, contrastada con RFC 2328)*.
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 §5.6 "Algorithms for routing": §5.6.3 el algoritmo de Dijkstra y el shortest-path tree, §5.6.4 OSPF.
- **`RFC 2328`** — OSPF, *Internet Standard* (1998): los cinco mensajes, los tipos de LSA, las áreas, la elección de DR/BDR, el flooding fiable.
- **`RFC 5340`** — OSPFv3 (2009), para IPv6 · **`RFC 5308`** — extensiones de IS-IS para IPv6.
- **`RFC 8402`** — arquitectura de Segment Routing (2018) · **`RFC 8660`** — SR-MPLS · **`RFC 8986`** — SRv6.
- **`RFC 9552`** — BGP-LS (dic-2023), obsoleta `RFC 7752` · **`RFC 7432`** — EVPN · **`RFC 7348`** — VXLAN.
- **`RFC 5797`** — TWAMP: medir ancho de banda sin carga intrusiva.
- **Clase anterior:** 32 — Enrutamiento distance-vector: Bellman-Ford, conteo a infinito y RIP
- **Clase siguiente:** 34 — BGP y el enrutamiento entre operadores